

Web Traffic Time Series Forecasting & Anomaly Detection

Time Series Analysis - Group Project Report

Dataset: Kaggle Web Traffic Time Series Forecasting

Daily views of 145,063 Wikipedia pages, 2015-07-01 to 2017-09-10

Part	Member	Scope
Part 1	Otmane ZAIM	Data Exploration & Decomposition
Part 2	Aymane ZEROUAL	Forecasting & Anomaly Detection
Part 3	Omar TANJI	Cleaning, Re-forecasting & Conclusions

GitHub repository: github.com/AyZee100/web-traffic-forecasting

Introduction

This project applies the complete time-series analysis workflow to real web traffic data: exploration and stationarity testing, decomposition, forecasting with two model families (SARIMA and Prophet), anomaly detection with four independent methods, and a controlled before/after experiment measuring how anomaly cleaning affects forecast accuracy.

The dataset. The raw Kaggle file contains one row per Wikipedia page (145,063 pages) and one column per day (803 days). Each cell is the number of views of that page on that day. Individual pages are sparse and noisy: most receive few views and have many missing days.

The key design decision: aggregation. Fitting a model to 145,063 separate noisy series is neither practical nor informative. We therefore summed all pages per day, producing a single series: total Wikipedia traffic per day (mean ~185.5 million views). Summation cancels page-level noise and preserves exactly the shared structure a forecasting model needs: trend, weekly rhythm, and global events. It also resolves missing values by construction: a missing page simply contributes zero to the daily total, so the aggregate series has **zero missing days out of 803** - no imputation is required.

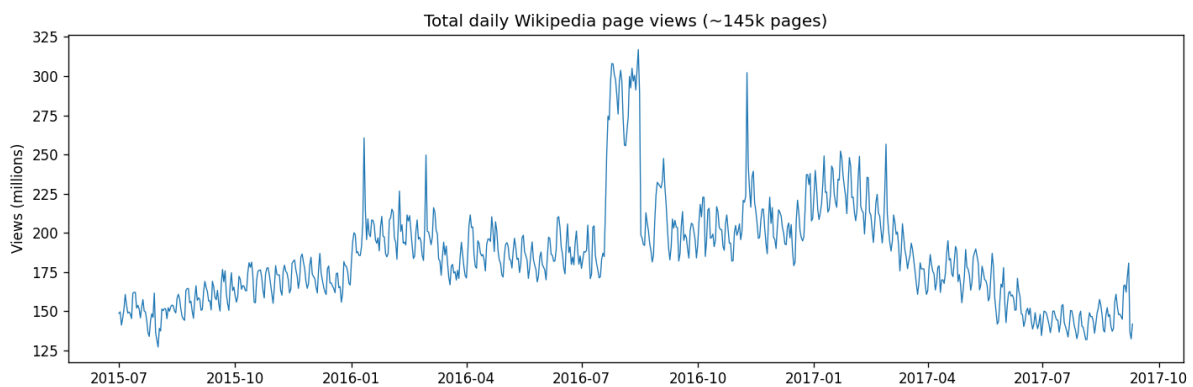


Figure 1 - Total daily Wikipedia page views (all 145k pages summed), 2015-2017.

Visible at a glance: a mild upward trend through 2016, a striking high-activity regime in July-August 2016, isolated one-day spikes, and a persistent short oscillation - the weekly cycle.

PART 1 - Otmane ZAIM - Data Exploration & Decomposition

1.1 Stationarity: the ADF test and differencing

A series is **stationary** when its statistical behaviour (mean level, variance) does not drift over time. The AR and MA components of ARIMA assume stationarity, so this must be checked first. The **Augmented Dickey-Fuller (ADF)** test does so formally: a p-value below 0.05 indicates stationarity.

Series tested	ADF statistic	p-value	Conclusion
Raw series	-2.655	0.082	non-stationary (borderline)
1st difference	-5.485	2.2e-06	stationary

The raw series fails the 5% threshold, consistent with its visible drift. (The raw-series result is borderline: it can flip below 0.05 in other library versions because the test automatically selects how many lags to include. The trend strength of 0.90 and the slowly decaying ACF both independently support treating it as drifting.) The remedy is **differencing**: analysing day-to-day changes instead of levels. Changes hover around a stable value even when the level climbs - and indeed the differenced series is unambiguously stationary. Hence **d = 1** in all subsequent models.

1.2 ACF / PACF: reading the correlation structure

The **ACF** (autocorrelation function) measures how similar each day is to the day k days earlier, including indirect echoes; the **PACF** (partial ACF) isolates only the direct link, with intermediate days factored out. Together they guide the choice of ARIMA orders.

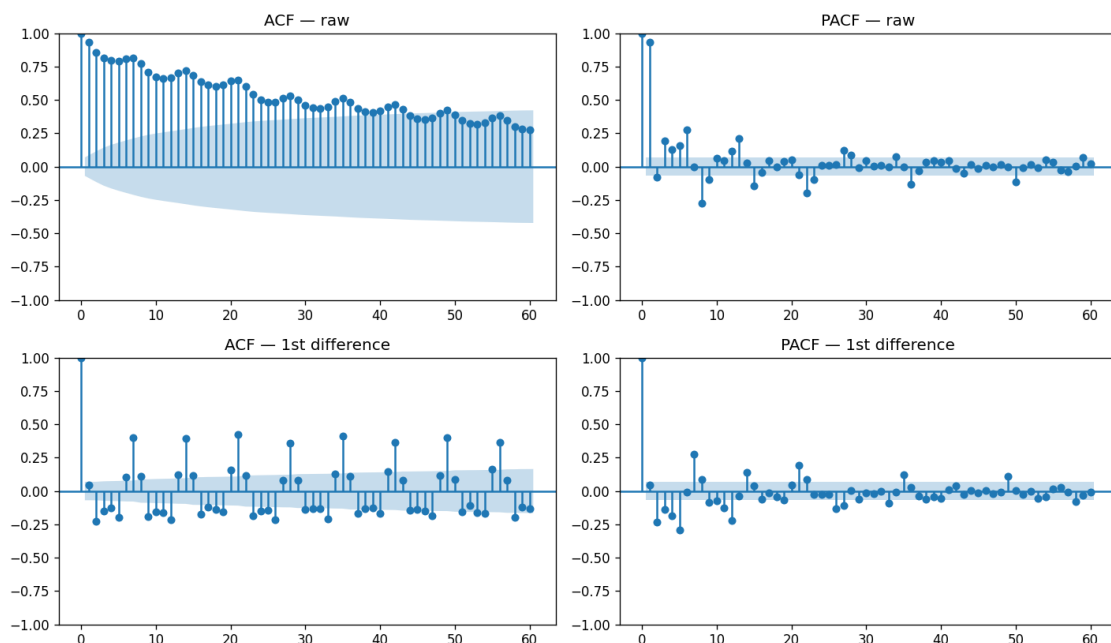


Figure 2 - ACF and PACF of the raw (top) and differenced (bottom) series.

Two readings matter. First, the raw ACF decays very slowly - a second signature of non-stationarity. Second, the differenced ACF shows sharp spikes at lags 7, 14 and 21: every day resembles the same weekday one week earlier. This weekly cycle dictates a **seasonal** ARIMA with period $m = 7$.

1.3 Decomposition: classical and STL

Decomposition splits the series into three additive layers: **trend** (long-term direction), **seasonality** (repeating weekly pattern) and **residual** (what remains). We applied both the classical moving-average method and **STL** (Seasonal-Trend decomposition using Loess). STL fits local regressions instead of rigid averages, allows the seasonal pattern to evolve, and - crucially - was run in **robust** mode, which prevents extreme days from bending the trend estimate.

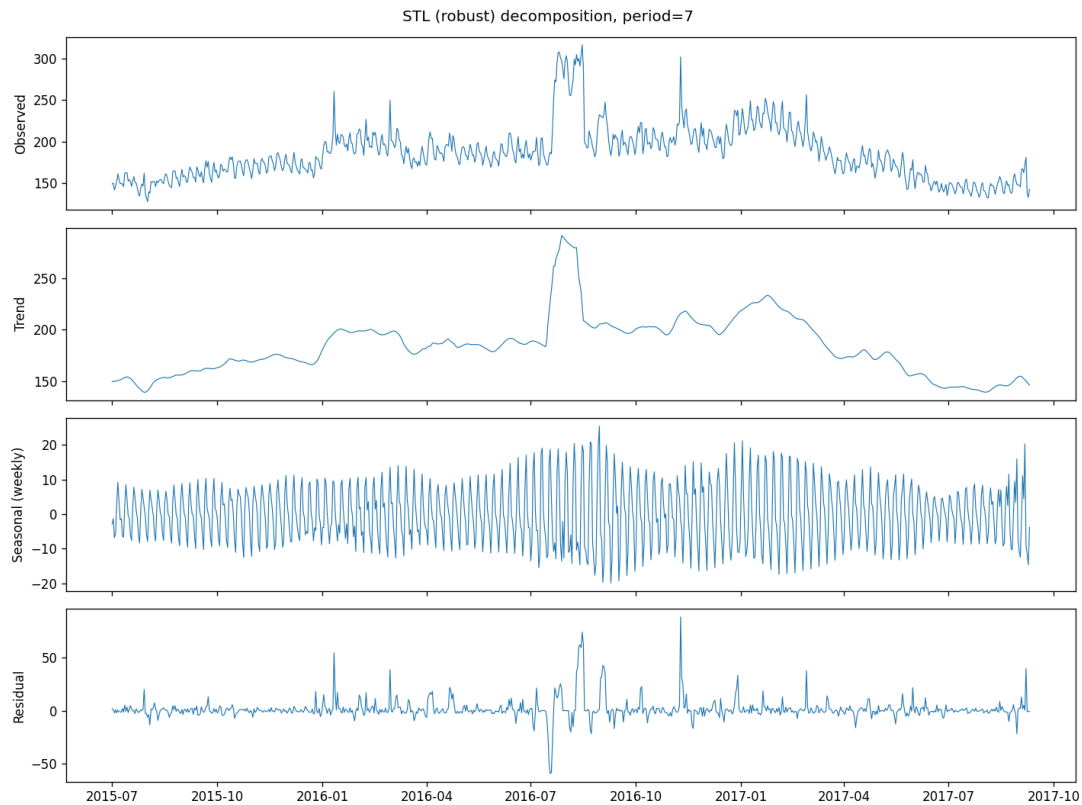


Figure 3 - Robust STL decomposition (period 7): observed, trend, weekly seasonality, residual.

Interpretation. The trend (strength 0.90) climbs from ~150M views/day in 2015 to a plateau near 190M in 2017. The weekly component (strength 0.37) is stable: Mondays peak at +11M versus an average day, Fridays dip to -10M - a classic workweek browsing pattern. The residual is small noise (sigma ~ 10M) punctuated by large isolated spikes. Because robust STL keeps those spikes out of the trend, they remain fully exposed in the residual - precisely where Part 2 hunts for anomalies.

PART 2 - Aymane ZEROUAL - Forecasting & Anomaly Detection

2.1 Experimental setup

The last 60 days (2017-07-13 to 2017-09-10) are held out as a test set the models never see during training. Forecast quality is measured by **RMSE** (root mean squared error - typical miss, large errors penalised extra) and **MAE** (mean absolute error), both in millions of views.

2.2 SARIMA: order selection by AIC

$\text{SARIMA}(p,d,q)(P,D,Q)_m$ applies the ARIMA ingredients twice: at the daily scale (predict from recent values and errors) and at the weekly scale (predict this Monday from last Monday). Rather than guessing orders, we grid-searched 15 configurations and ranked them by **AIC**, which rewards fit but penalises complexity to prevent overfitting. Winner: **SARIMA(1,1,2)(0,1,1)₇** (AIC 5316.9) - one difference at each scale, exactly matching the Part 1 diagnostics.

2.3 SARIMA vs Prophet

Prophet (Meta) fits a single global curve - piecewise trend + yearly cycle + weekly cycle - through the entire history and extrapolates it. The two models thus embody opposite philosophies: SARIMA is local and short-memory; Prophet is global and curve-fitting.

Model	RMSE (M)	MAE (M)	Relative error	AIC / BIC
SARIMA(1,1,2)(0,1,1) ₇	12.65	8.53	~7%	5316.9 / 5339.9
Prophet	38.11	32.38	~20%	-

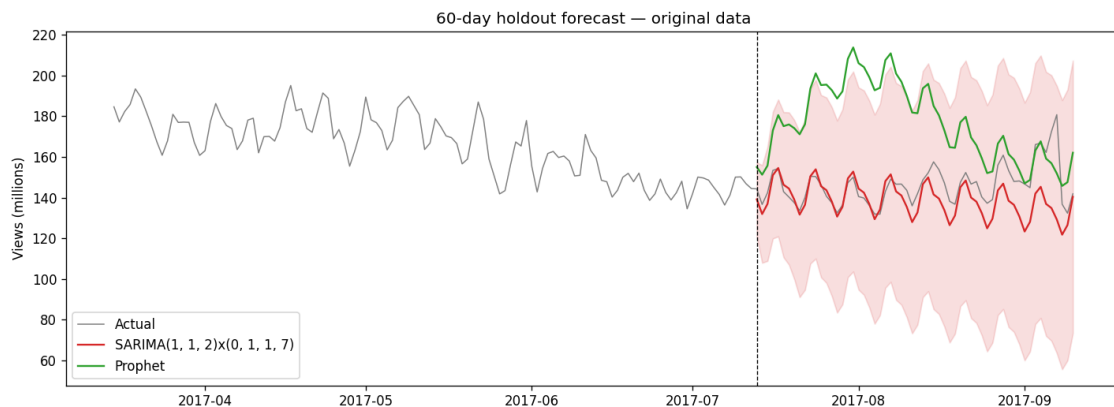


Figure 4 - 60-day holdout forecast on original data. SARIMA (red) tracks reality; Prophet (green) overshoots.

SARIMA wins by a factor of three. The anomalous mid-2016 regime (Olympics, election campaign) bent Prophet's global trend and yearly-seasonality estimates, so its extrapolation into late 2017 overshoots. SARIMA, anchored by differencing to the recent level, is unaffected.

2.4 Anomaly detection: four independent methods

An anomaly is a day that deviates strongly from the pattern implied by trend + weekly seasonality. Four detectors, from four families, each answering "what is abnormal?" differently:

1. Robust Z-score on STL residuals (statistical). Distance from typical, in MAD-based deviations; $|z| > 3.5$ flags a day. The most sensitive: **99 flags**.

2. SARIMA standardized residuals (statistical). Days where the one-step-ahead forecast is badly surprised ($|\text{error}| > 3 \text{ sd}$). Precise: **13 flags**; doubles as a real-time alerting signal.

3. Isolation Forest (machine learning). Random binary splits on features [level, 1-day change, 7-day change, STL residual, weekday]; days isolated in few cuts are outliers. **17 flags**.

4. LSTM autoencoder (deep learning, optional extension). A recurrent network trained to compress and reconstruct 14-day windows learns only normal weeks; high reconstruction error exposes abnormal ones. **8 flags**. (Not bit-reproducible across environments; counts may shift slightly.)

Consensus voting. No single detector is trusted alone - the z-score is over-sensitive, others may miss context. A day is retained only if flagged by **at least 2 of the 4 methods**: 20 consensus anomalies.

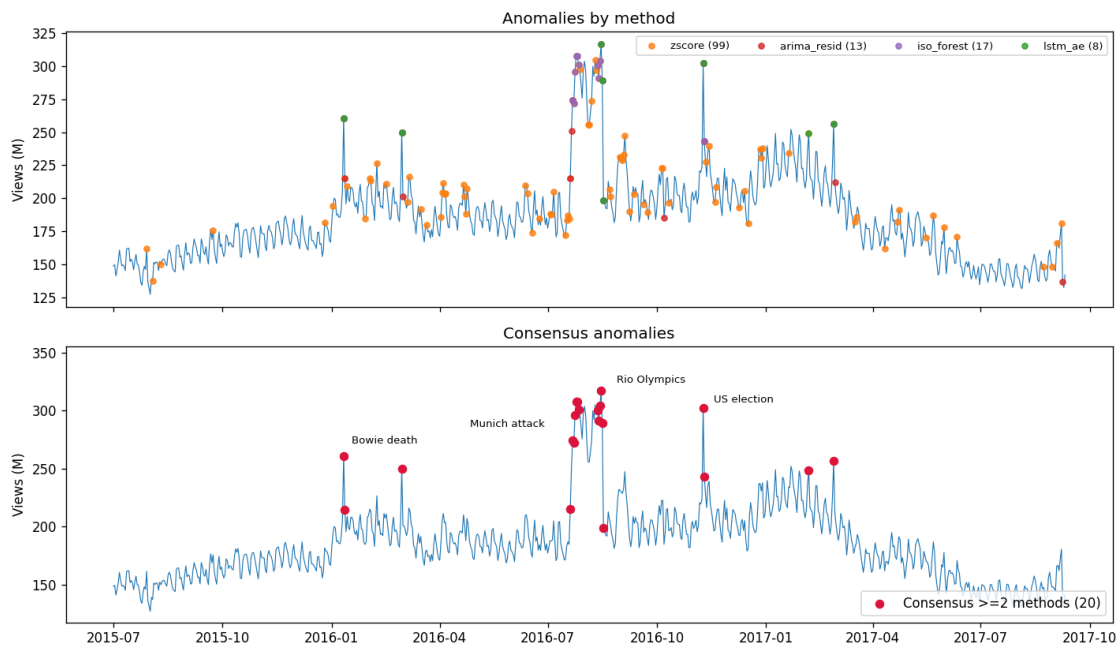


Figure 5 - Top: flags per method. Bottom: the 20 consensus anomalies with event labels.

Date(s)	Real-world event
2016-01-11 / 12	Death of David Bowie
2016-02-29	Leap day
2016-07-20 to 2016-08-17	Munich attack, US conventions, Rio Olympics (all-time peak: 316.9M on Aug 15)
2016-11-09 / 10	US presidential election result (302M views)
2017-02-06 / 2017-02-27	Super Bowl LI / Oscars Best-Picture mix-up

Every consensus anomaly maps to a nameable event - the strongest possible validation that the pipeline detects reality, not artefacts. These spikes inflate variance estimates, widen prediction intervals, and distort the trend/seasonality estimates of global-fit models - measured directly in Part 3.

PART 3 - Omar TANJI - Cleaning, Re-forecasting & Conclusions

3.1 Seasonality-aware imputation

Anomalous days cannot simply be deleted - that would break the calendar and the weekly cycle. Instead, each of the 20 consensus anomalies is **imputed**: replaced by the median of the **same weekday** within +/- 4 weeks. A spiked Tuesday becomes a typical nearby Tuesday, preserving both the weekly pattern and the local trend level.

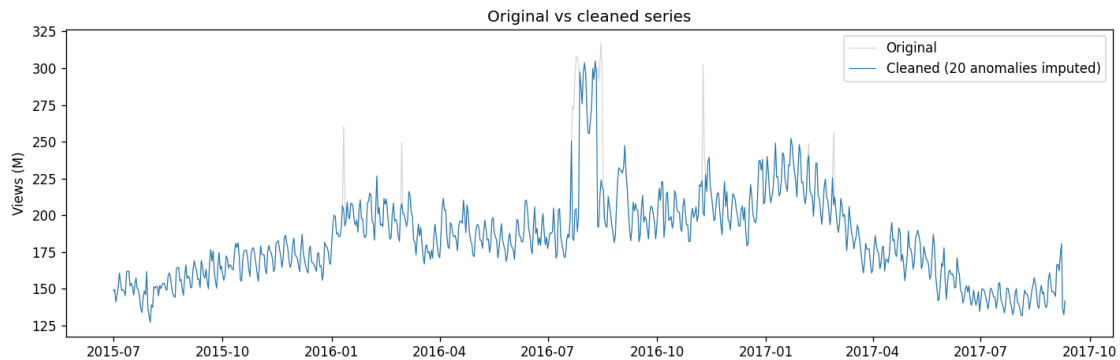


Figure 6 - Original series (grey) vs cleaned series (blue): spikes removed, structure intact.

Fair evaluation protocol. Both models are retrained on the cleaned history but scored against the **original, uncleaned** test data - real future traffic is what a forecaster must predict. All 20 anomalies fall before the forecast origin, so the test set itself is untouched and the comparison is fair.

3.2 Re-forecast: before vs after cleaning

Model	RMSE before	RMSE after	MAE before	MAE after	Change
SARIMA(1,1,2)(0,1,1)7	12.65	16.06	8.53	11.68	slightly worse
Prophet	38.11	22.16	32.38	17.81	-42% (better)

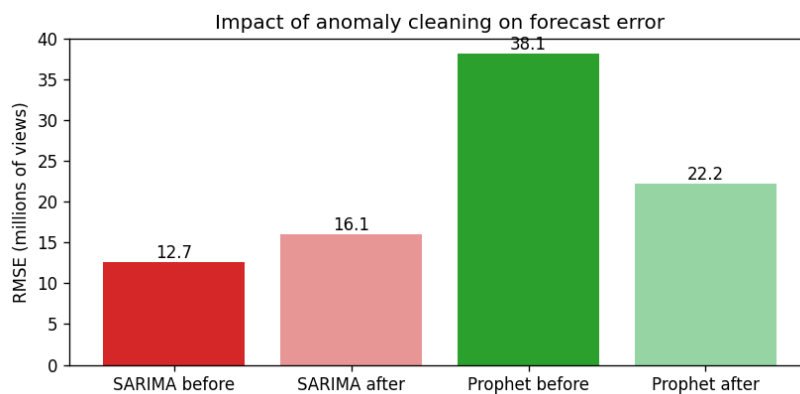


Figure 7 - Holdout RMSE before (solid) and after (light) anomaly cleaning.

The asymmetry is the central finding. Prophet improves dramatically (-42%): with the spikes gone, its global trend and yearly-seasonality estimates are no longer distorted, and its extrapolation finally points the right way. SARIMA gets *slightly worse*: it had already absorbed the spikes through differencing, and imputing them altered the variance structure its parameters were calibrated on. Anomaly handling is therefore **model-dependent**: essential for global curve-fitters, unnecessary - and potentially mildly harmful - for short-memory differencing models. Note also that even after

cleaning, Prophet (22.16M) does not catch SARIMA on original data (12.65M).

3.3 Conclusions and recommendations

1. SARIMA is the right operational model for this series: best accuracy (~7% relative error), inherent robustness to anomalies, and interpretable orders backed by the ADF test, ACF/PACF and AIC.

2. Clean history for global curve-fitters, not for differencing models. The 42% Prophet improvement versus the slight SARIMA degradation demonstrates this directly.

3. Use consensus voting for anomaly detection. Individual detectors are noisy (99 z-score flags); requiring agreement of at least 2 of 4 methods produced 20 precise, fully explainable events.

4. Treat event spikes as signal, not garbage. Elections, sports finals and celebrity deaths are predictable in kind. The production-grade approach is not deletion but SARIMAX or Prophet with event/holiday regressors, so the model anticipates such days instead of being surprised by them. SARIMA's standardized residuals additionally provide a free real-time anomaly alarm.

Reproducibility

The GitHub repository contains the executed notebook (*web_traffic_forecasting.ipynb*), the cached aggregate (*data/daily_total.csv*, 23 KB), all figures, the slide deck with a timed speaker script, and *requirements.txt*. Full re-run: `pip install -r requirements.txt`, then Run All (~3-5 min). Re-aggregation from the raw 400 MB Kaggle file is automatic if the cache is removed.